

T4D AI COHORT 2



Building Generative AI Workflows:

Our journey building Avni AI App configurator

The Problem

Why a chat-first AI configurator for Avni

Manual webapp config

Click-by-click through Avni UI.
Avni expertise required. Consistency on the user.

IDE-based AI (Cursor, Claude Code)

Developer-only. Decoupled from Avni.
Quality varies with AI + user effort.

Our AI App Configurator — chat-first, not click-first

- Chat-first. No webapp, no code, no Avni expert.
- Program managers drive the setup themselves..
- Schema validation + guardrails in the flow, not LLM judgment.
- Deterministic pipeline → reproducible, auditable bundles.
- Earlier it used to take few hours for average form creation, now that gets done in few minutes
- Usage is high during initial setup and sporadic usage later on

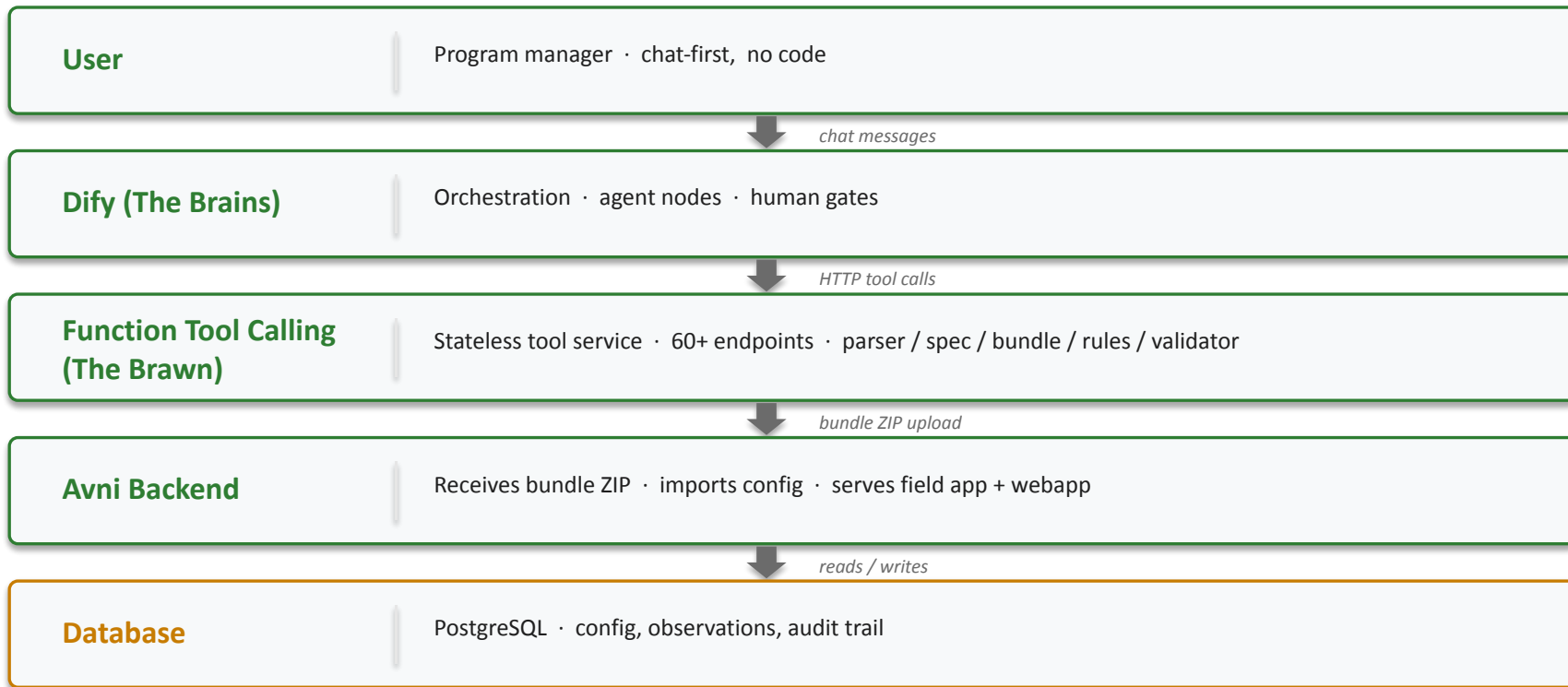
Looks like an LLM problem. Most of it is deterministic.



Avni App Configuration complexity

What We Built

Tier architecture — user chat down to the database



Deterministic on the rails, agentic in the gaps.



Avni App Configurator Demo

Lesson 1 — WHERE to put the LLM

System-level architecture · deterministic code + LLM agents + variables

Write the contract first. Writable → deterministic. Fuzzy → scoped agent.

Go deterministic when

- Schema is writable
- Reproducibility matters
- Validation expressible using logic
- LLM cost not justified

e.g. parsing · validation · state management

Use an LLM agent when

- Input shape varies
- Judgment or reasoning needed
- User dialog / clarification
- Open-ended synthesis or correction

e.g. classification · dialog · generation · auditing

THE GLUE — variables thread state between the two sides

session ids · phase flags · approvals · retry counters · pause signals

Can write the contract → deterministic. Can't → agent.

Avni Spec Configuration Template

```
# Registration form
registrationForm:
  decisionRule: "..." # Form-level decision rule
  visitScheduleRule: "..." # Form-level visit scheduling
  validationRule: "..." # Form-level validation
  checklistsRule: "..." # Form-level checklist rule
  sections:
    - name: "Basic Details"
      fields:
        - name: "Field Name"
          dataType: Text # Text|Numeric|Coded|Date|DateTime|Duration|PhoneNumber|Audio|Image|ImageV2|Video|File|Location|Subject|GroupAffiliation|Id|Time|Notes|QuestionGroup
          selectionType: Single # Single | Multi (for Coded fields)
          mandatory: true
          # Numeric-specific
          min: 0 # lowAbsolute
          max: 100 # hiAbsolute
          lowNormal: 10 # Normal range lower bound
          hiNormal: 90 # Normal range upper bound
          unit: "kg"
          # Coded-specific
          options: ["Option A", "Option B"]
          # Skip logic
          skipLogic:
            dependsOn: "Other Field"
            condition: "="
            value: "Yes"
          # Key-value metadata
          keyValues:
            editable: false
            ExcludedAnswers: ["uuid1"]
          # Validation format
          validFormat:
            regex: "^[6-9]\\d{9}$"
            descriptionKey: "Must be 10 digit mobile number"
          # Element-level rule (JavaScript)
```

Lesson 2 — HOW to shape the agents

Agent-level design · sub-agents, divide and conquer

One big agent → specialist sub-agents. Smaller brain. Clearer job. Surgical edits.

Small context

Each agent sees only its slice.

Low tokens

Small contexts × many agents
= cheap. Caching wins.

One specialist, one job

Clear I/O contract.
Bounded iterations.

Surgical edits

Edit one file, one rule.
Never regenerate the whole.

Common sub-agent archetypes

Classifier

Route input to
the right path.

Planner

Break task into
ordered steps.

Synthesizer

Produce the
artifact as per the spec.

Patcher

Edit one part.
Not regenerate.

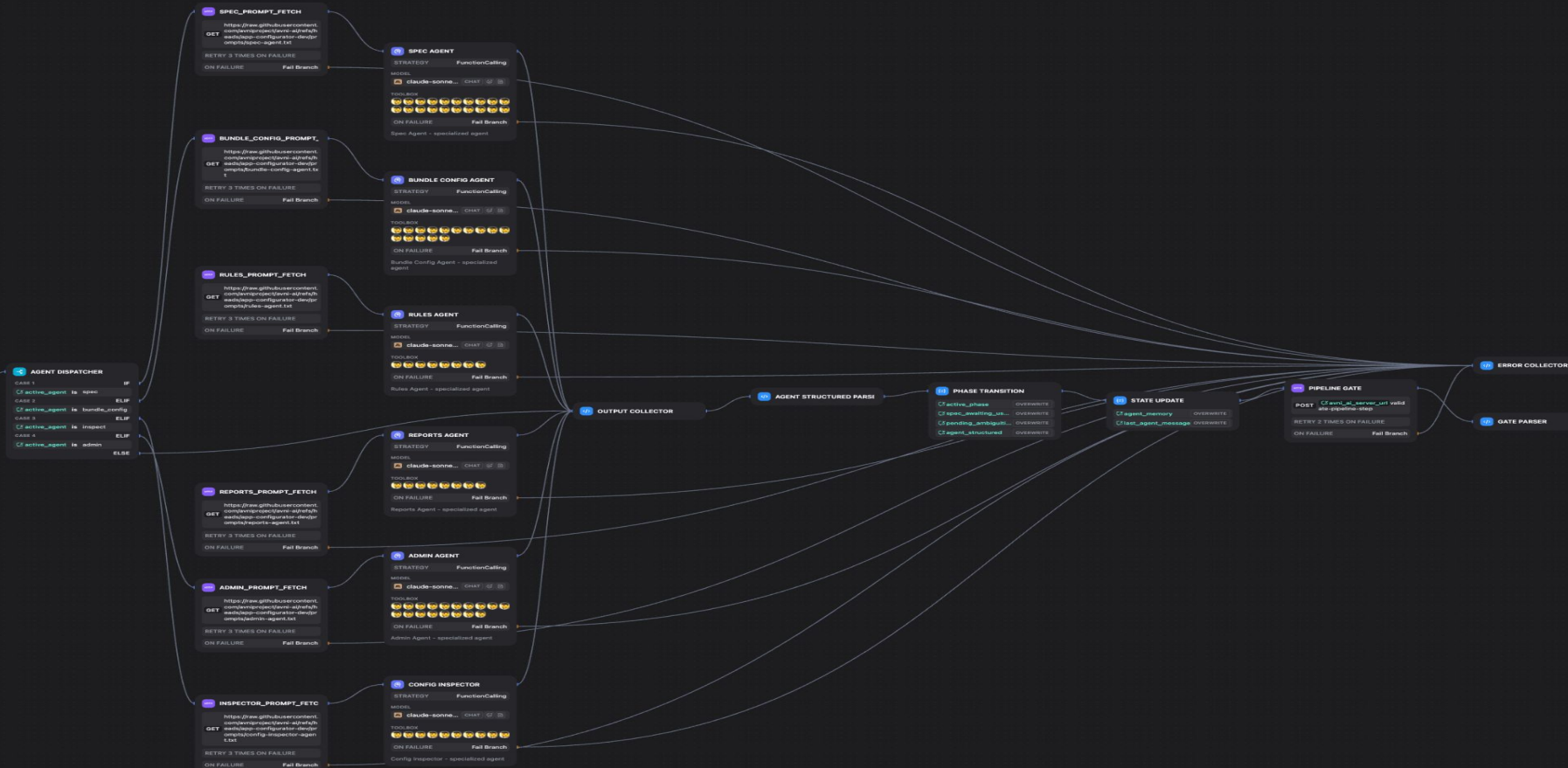
Inspector

Audit output.
Read-only critic.

Small brain. Clear job. Surgical edits.

Avni App Configurator Agentic Workflow

AGENT LOGS



Lesson 3 — HOW to design the tools

Tool-level design · keeps the LLM cheap, fast, capable of depth

Design every tool assuming an LLM will call it. Return only what's needed.

Sectional retrieval

Return only what this turn needs. Not the whole base.

Response truncation

Cap outputs.
Send 'top 20 + count'.

Surgical edits

Target + patch.
Not full rewrite.

Stateless

Individual tools do not maintain state within themselves.
But you could have tools to save, modify or retrieve overall state(data).

Handles, not payloads

Heavy state in a store.
LLM sees a handle + summary.

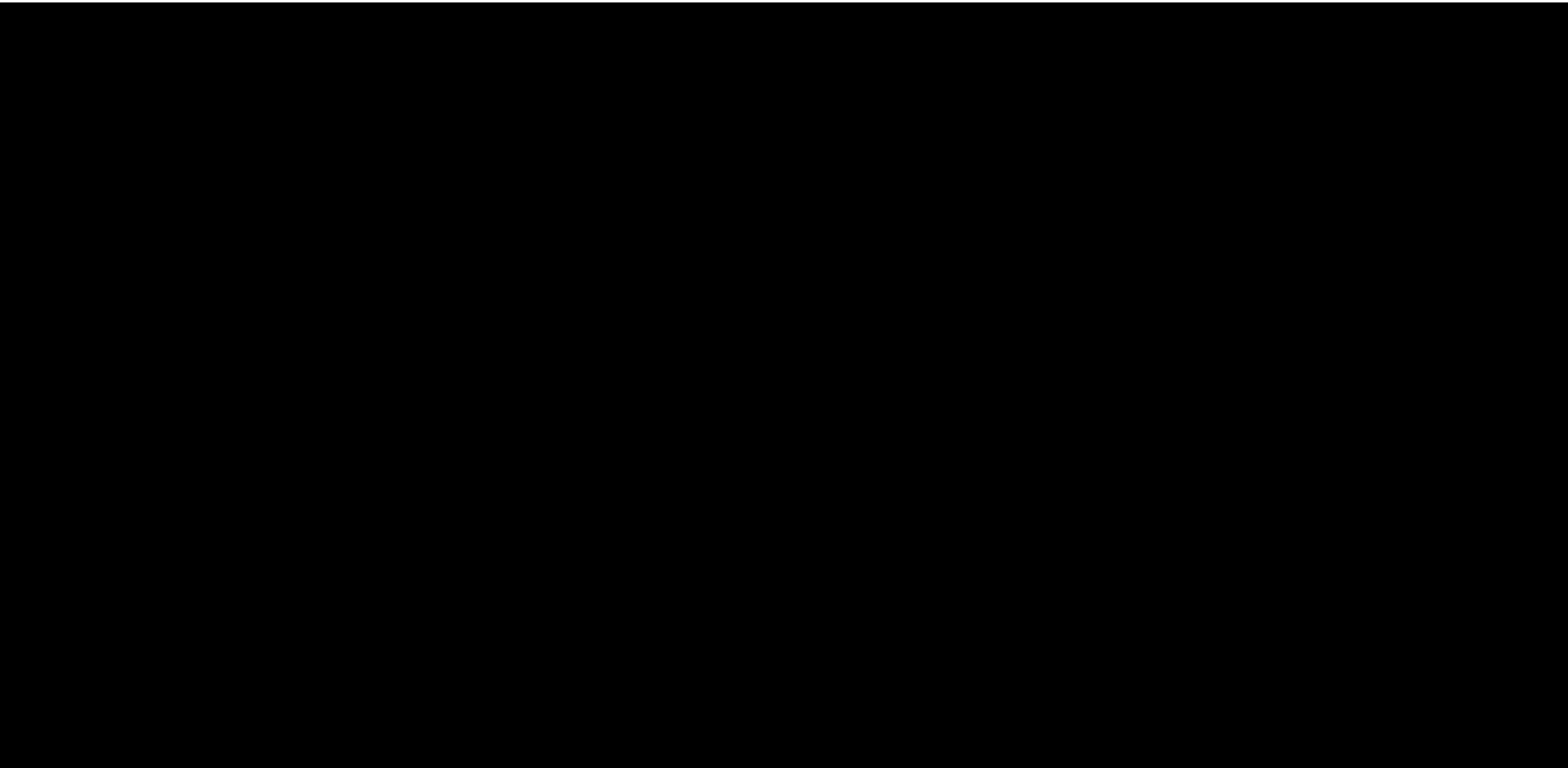
Defensive guards

Validate in, validate out.
Bound depth. Fail clean.

Before — LLM exhausted on initial entities. Couldn't go deeper.

With these — 10× fewer tokens per turn. Depth becomes routine.

Design the tool for the Agent that will call it.



What's Next, and What's in the Way

Roadmap · Dify workflow shortcomings

Orchestrating open-ended long-horizon tasks using LangChain/LangGraph/Dify was a lost cause for us. Moving to Claude-managed agents instead.

Shipping next

- Complete the bundle — enhance support for complex configuration
- Handle multi-turn conversation for configuration refinement
- Production org safety — trial-only today
- Adopt cohort guardrails — regex + small-LLM guards
- Benchmark setup time vs. manual webapp config
- Unstructured scoping docs (PDF / image) → spec

Dify workflow — shortcomings

- Doc extractor misses content — parsing moved to our own parser on avni-ai
- Agent nodes can't call auth'd tools using perishable tokens — token lives in conversation-scoped in-memory store
- No sub-workflow imports — HTTP-self (200–500 ms) or copy-paste across flows
- Prompt length UI limits — inject long prompts as variables
- YAML / visual-flow config — adoption curve for developers used to code files

The Devil is in the details.!

THE NEXT CHAPTER · Take 2 - Side A



avni-skills-sdk

Claude-managed agents

“Claude Code, but for an Avni bundle generation”

The New Architecture — two stages, one workflow

System-level · deterministic generator + Claude Agent SDK refinement loop

Forms.xlsx + Modelling.xlsx in. Ready-for-AVNI bundle out. The LLM only refines.

Stage 1 · Deterministic

- No LLM · < 1 s · free
- 2 Excel files → 30+ JSON + zip
- Reproducible, auditable bundles
- Runs the validator, not a model

e.g. parse · generate · validate

Stage 2 · Chat-driven refinement

- Claude Agent SDK loop
- Each turn becomes a git commit
- Validator runs after every turn
- Correct, extend, ship — in chat

e.g. fix C5 · add subject · rename

THE BRIDGE — slide 12's "moving to Claude-managed agents," realized

deterministic rails stay free & fast · agentic judgement now audited, capped, reversible

Same thesis as before — now without Dify.

Multi-step build — tools run before every edit

cost\$0.1837tokens in/out74/347615:00 [69/544]

[stream end · 32 events]

you > session should be group , form should contains session id , session name and session date

agent > haiku-4-5

claude-haiku-4-5 · cwd=sess_0a21330c5ae3aedd/bundle

33 tools · 8 skills available

agent >

Perfect! Now I understand. Session should be a group type with a registration form containing session id, session name, and session date. Let me build this step by step:

Step 1: Add the "Session" subject type

First, I'll add Session to subjectTypes.json, modeling it after Cohort (which is also a group):

o Read bundle/subjectTypes.json

↳ 1 [· +41 lines

o Edit bundle/subjectTypes.json

↳ The file /Users/samanvay/.avni-skills-sdk/sessions/sess_0a21330c5ae3aedd/bundle/subjectTypes.json

agent >

Step 2: Add concepts for session fields

Now I need to create concepts for the three fields. Let me check if they exist:

o mcp_avni-bundle_bundle_find_concept

↳ { · +5 lines

o mcp_avni-bundle_bundle_find_concept

↳ { · +5 lines

o mcp_avni-bundle_bundle_find_concept

↳ { · +5 lines

agent >

Perfect! All three concepts are safe to add. Let me read the end of concepts.json to add them:

o Read bundle/concepts.json

↳ 720 "active": true · +49 lines

\$ Bash tail -20 /Users/samanvay/.avni-skills-sdk/sessions/sess_0a21330c

↳ "order": 1 · +19 lines

\$ Bash tail -50 /Users/samanvay/.avni-skills-sdk/sessions/sess_0a21330c

↳ "name": "Fear judgement", · +49 lines

o Edit bundle/concepts.json

↳ The file /Users/samanvay/.avni-skills-sdk/sessions/sess_0a21330c5ae3aedd/bundle/concepts.json has

agent >

Step 3: Create the registration form for Session

Now I'll create the registration form. Let me check an existing form structure first:

\$ Bash ls -la /Users/samanvay/.avni-skills-sdk/sessions/sess_0a21330c5a

↳ total 440 · +19 lines

o Read forms/Cohort Registration_1de0549d-a83f-6c9e-bca6-94f64d421288.json

↳ 1 { · +99 lines

agent >

Perfect! Now I'll create the Session Registration form:

o Write forms/Session Registration_d9e8f7a6-c5b4-4a3b-8c2d-1e9f7a6b5d4c.json

↳ File created successfully at: /Users/samanvay/.avni-skills-sdk/sessions/sess_0a21330c5ae3aedd/bu

agent >

Step 4: Add form mapping for the Session Registration form

Now I need to add the form mapping. Let me check the current form mappings:

o Read bundle/formMappings.json

avni-sdk-0:[tmux]*

per-agent

agent	schema	abort	cost	turns	ok
agent		1	1	0	

wallet · \$0.0937 / \$5.00 (1.9%)

2%

session

server unreachable

@ http://localhost:3030

start the REPL with

npm start in the

left pane.

failures

no failures yet

0 schema errors

0 circuit breaks

0 agent errors

0 validator regress

0 integrity issues

0 semantic failures

0 ambiguity loops

recent steps

SIDDHARTH-HARSH-RAJ-9 15:05 26-May-26

Building block 1 — spec in, bundle out, round-trip

Stage 1 · deterministic · avni-skills

One human-readable YAML spec compiles to a full Avni bundle — and decompiles back. Edits patch the live bundle; they never regenerate it.

Spec → bundle

YAML parses to entities;
generator emits 30+ JSON files.

Bundle → spec

Emitter reverses it; parse→emit
is byte-equivalent.

Patch, don't regen

New spec merges onto the live
bundle — UUIDs + UI edits kept.

Deterministic IDs

Hash-seeded UUIDs. Same spec
→ same bundle, every run.

Canonical ZIP order

Files in server load order.
No silent import failures.

21-org schema

Reverse-engineered from 21
prod orgs. Zero missed keys.

Before — bundles hand-built in the UI or regenerated from scratch; edits and UUIDs lost.

Now — a versioned spec compiles, round-trips, and patches in place.

The bundle is a build artifact of a spec — not a hand-edited blob.

Building block 2 — bundle must adhere to a graph

Stage 1 · deterministic · avni-skills

Model the bundle as a UUID graph of foreign keys. Then you can ask what depends on what — and catch broken references before the server ever sees them.

Every entity a node

Concepts, forms, programs, mappings — keyed by UUID.

Every link an edge

Typed references: form element
→ concept, mapping → program.

Impact check

What breaks if I delete this?
See it first.

Prerequisites

What must already exist
for this to be valid?

Broken-link scan

Flags every reference
to a missing UUID.

Form-type aware

Knows a program encounter
needs a program + type.

Before — dangling references slipped through and failed silently on upload.

Now — integrity is checked locally; delete-impact is visible before you act.

Know what breaks before it breaks.

Building block 3 — knowledge the agent can actually use

Stage 1 · Knowledge corpus · avni-readme · avni-ai · avni-skills

Three sources — implementer docs, a merged corpus, and patterns mined from real bundles — compressed into chunked, tagged knowledge the agent pulls on demand.

Docs as source

203 implementer guides
(~21k lines) from avni-readme.

Merged corpus

Assembled into merged.md —
~557 KB, ~22.5k lines.

Compressed + chunked

Split into 68 topic files
with semantic chunks.

Metadata-tagged

Keywords, audience,
retrieval-boost per chunk.

Patterns from prod

4,949 concepts + 132 rule
templates mined from real orgs.

Retrieval-ready

Agent pulls the right topic,
not the whole monolith.

Before — one 557 KB monolith; retrieval dragged in noise — ~half was infra/reporting.

Now — 68 focused, tagged topics (~96% implementer) + mined patterns.

Compress for retrieval — high signal beats raw volume.

Validator-state injection + :model switching mid-session

```
→ Religion: answer "other" → 5ec55e95-0f4d-02f6-43f3-cbe91452443c
W datatype-mismatch forms/* (forms: Cohort Registration)
→ Donor: form says Text, concepts.json says Coded
W trailing-whitespace encounterTypes.json
→ Cohort Endline
W trailing-whitespace forms/*.json
→ Cohort Endline

you > :model sonnet
✓ model set to claude-sonnet-4-6
(sonnet recommended for structural fixes - case-insensitive concept reuse, schema decisions)

you > fix all the errors
— agent · sonnet-4-6 —
agent call failed: 429 — {"error": "rate limit", "retryAfterMs": 223}

you > :model haiku
✓ model set to claude-haiku-4-5-20251001
(haiku is cheap+fast for mechanical edits and rule authoring; switch to sonnet for structural fixes)

you > fix all the errors
— agent · haiku-4-5 —
agent call failed: 429 — {"error": "rate limit", "retryAfterMs": 640}

you > fix all the errors
— agent · haiku-4-5 —
claude-haiku-4-5 · cwd=sess_0a21330c5ae3aeddbundle
33 tools · 8 skills available
agent >
Looking at the CURRENT VALIDATOR STATE, there is **1 error**:

- **C5** Answer "other" (UUID: 5ec55e95-0f4d-02f6-43f3-cbe91452443c) in concept "Religion" not found as standalone concept

This is a **CODED ANSWERS** violation (HARD RULE #3): the "other" answer exists inside the Religion concept but is missing as a standalone concept in concepts.json.

Let me read concepts.json to understand the structure:
♦ Read /concepts.json
↳ File does not exist. Note: your current working directory is /Users/samanvay/.avni-skills-sdk/se
$ Bash ls -la
↳ total 272 · +21 lines
♦ Read bundle/concepts.json
↳ File content (29053 tokens) exceeds maximum allowed tokens (25000). Use offset and limit paramet
♦ Grep Religion
↳ 1202- "uuid": "0b69cf24-3a38-16be-889b-df838bcf7f32", · +10 lines
♦ Grep Religion
↳ 1187- "answers": [ · +40 lines
♦ Grep other
↳ 1235- "order": 4 · +14 lines
agent >
I found the issue. The Religion concept has an answer "other" with UUID '5ec55e95-0f4d-02f6-43f3-cbe91452443c', but this UUID doesn't exist as a standalone concept in concepts.json.

Per **HARD RULE #3** (CODED ANSWERS), every answer in a Coded concept must exist as a standalone concept. Let me use the mandatory concept-lookup gate before creating it:
[avni-sdk-0:[tmux]]
```

14:50 [424/544]

per-agent

agent	turns ok
schema abort	cost
agent	1 1 0

wallet · \$0.0937 / \$5.00
(1.9%)

2%

session

server unreachable
@ http://localhost:3030

start the REPL with
npm start in the
left pane.

failures

no failures yet

- 0 schema errors
- 0 circuit breaks
- 0 agent errors
- 0 validator regress
- 0 integrity issues
- 0 semantic failures
- 0 ambiguity loops

recent steps

Building block 4 — logged, gated, capped

Stage 2 · *agentic SDK* · *avni-skills-sdk*

Assume the agent will try everything. Make every turn recorded, every destructive path blocked, and every dollar capped — by construction, not by trust.

Git-backed history

One commit per turn — a git repo.
Diff, revert, or resume any turn.

Audit by construction

Validator state in every prompt;
self-corrects on resume; durable.

Destructive ops gated

Hook blocks git / rm -rf / sudo;
a detector reverts foreign commits.

Cost capped

Hard wallet: \$5/session, \$1/turn;
aborts a no-edit token spree.

Before — state lived in chat memory; an agent could wander or burn the budget.

Now — every turn is git-backed and auditable; destructive paths gated; spend capped.

Audit-rigorous and bounded — by construction.

Regression detection + :eval LLM semantic audit

```
changed concepts.json
validator ✖ 1 errors · 1 warnings ? :1 Δ 0
A regression new?
rules ✓ clean
cost $0.0937 tokens in/out 82/3449
```

⚠ The agent may have claimed success. Consider :diff 1 then :revert 0 – or :model sonnet before re-attempting.
[stream end · 35 events]

you > :eval

running LLM semantic-audit (~\$0.05-0.20)...

✓ evaluated · cost \$0.0654 · 5 findings

The bundle has a critical architecture break: 6 Encounter forms exist but programs.json is empty, making these forms operationally unusable and leaving 18 formMappings orphaned. Beyond this blocker, the concepts layer exhibits semantic pollution (staff names mixed in, 59% of concepts are orphaned NA-type with zero answers), and incomplete verification of registration form coverage for both subject types. The missing programs array must be resolved before any deployment.

E No Programs defined but 6 Encounter-type forms exist

evidence: programs.json is empty (count: 0); forms.json contains 6 Encounter forms: 'Baseline for Men', 'Baseline for Women', and variants

recommend: Create program entries in programs.json for each program these encounters belong to (e.g., 'Active Bystander Program'), then establish formMappings linking encounters to programs.

E FormMappings reference orphaned or non-existent programs

evidence: 18 formMappings exist but programs array is empty; formMapping entries with programId references cannot resolve to valid programs

recommend: Regenerate formMappings after creating programs.json entries; validate that all programId foreign keys in formMappings.json resolve to actual program UUIDs.

W Personal staff names stored as semantic concepts

evidence: concepts.json includes 'Thejaswini H', 'Namitha Tresa', 'Sumathi Raj', 'Priya Vardarajan' as dataType NA concepts with zero answers

recommend: Move staff/facilitator names to a separate reference metadata file; concepts.json should contain only domain-level semantic entities.

W High semantic noise: 59% of concepts are NA-type with zero answers

evidence: 139 of 234 concepts have dataType NA and answerCount 0; examples 'Session', 'Meeting', 'Workshop' appear designed as answer options, not standalone concepts

recommend: Audit and reorganize: convert 'Session', 'Meeting', 'Workshop' to answers under a 'Session Type' coded concept; delete or archive unused NA-type concepts.

W Unverified registration form coverage for all subject types

evidence: 2 subjectTypes declared ('Cohort', 'Participant'); 2 IndividualProfile forms exist; sample metadata does not confirm both types have a registration form

recommend: Audit formMappings to confirm each of 'Cohort' and 'Participant' has exactly one IndividualProfile form; verify subjectType references in the mapping are correct.

you > :files

52 files:

- .claude/skills/architecture-patterns
- .claude/skills/avni-implementer-reference
- .claude/skills/backend-architecture
- .claude/skills/data-migration
- .claude/skills/field-implementation
- .claude/skills/go-live-checklist

[avni-sdk-8:[tmux]]

per-agent

agent	turns	ok
schema	abort	cost
agent	1	1 0

wallet · \$0.0937 / \$5.00 (1.9%)

2%

session

server unreachable
@ http://localhost:3030

start the REPL with
npm start in the
left pane.

failures

no failures yet

- 0 schema errors
- 0 circuit breaks
- 0 agent errors
- 0 validator regress
- 0 integrity issues
- 0 semantic failures
- 0 ambiguity loops

recent steps

SIDDHARTH-HARSH-RAJ-9 15:05 26-May-26

Building block 5 — prefer the tools, observe everything

Stage 2 · *agentic SDK* · *avni-skills-sdk*

Give the agent purpose-built tools it reaches for instead of raw shell — and make everything it does observable, three ways.

Prefer tools over shell

Four in-process bundle tools:
validate, find-concept, summary, export.

Determinism on rails

Deterministic :summary handles routine;
LLM :eval audits the gaps.

Captured three ways

On-disk JSONL, live HTTP
endpoints, and REPL commands.

See it live

A live dashboard: per-agent
cost and failures by category.

Before — the agent improvised with shell; runs were opaque and hard to debug.

Now — it prefers a few bundle tools, and every action is logged, served, inspectable.

Deterministic checks on rails; LLM judgement only in the gaps.

One bundle, end to end · :zip export · \$0.13

```
2. **Added Session Subject Type**
  - Created new subject type "Session" (UUID: 9f7a2b1c-4d5e-47f8-9a3b-2c8d7e1a5f4b)
  - Configured as a group type with "Individual" entity type
  - Added to operationalSubjectTypes.json

3. **Added Session Concepts** (3 new concepts)
  - "Session Id" (Text)
  - "Session Name" (Text)
  - "Session Date" (Date)

4. **Created Session Registration Form**
  - Form UUID: d9e8f7a6-c5b4-4a3b-8c2d-1e9f7a6b5d4c
  - Contains three form elements for session id, name, and date
  - Form type: IndividualProfile

5. **Added Form Mapping**
  - Linked Session Registration form to Session subject type

**Validator Status:** ✅ **Valid** - 0 errors, 1 warning (unrelated "Donor" dataType mismatch)

The Session subject type is now ready for use in your AVNI bundle!
```

```
turn 3 · dbb71ae50a62

changed concepts.json, formMappings.json, operationalSubjectTypes.json, subjectTypes.json, forms/Session Registr
ation_d9e8f7a6-c5b4-4a3b-8c2d-1e9f7a6b5d4c.json
validator ✅ valid · 1 warnings Δ 0

rules ✅ clean

cost $0.1328 tokens in/out 132/6472
```

[stream end · 61 events]

```
you > :zip
✓ /Users/samanvay/Developer/avni-skills-sdk/output-bundle/Durga India-sess_0a21330c5ae3aedd.zip (55.2 KB)
```

```
you >
avni-sdk-0:node*
```

per-agent		wallet · \$0.3955 /
agent	turns ok	\$5.00 (7.9%)
schema	abort cost	
agent	3 3	8%
		session
		sess_0a21330c5ae3aedd
		turns: 3
		cost: \$0.3955 /
		\$5.00
		tokens: in 306 ·
		out 17028
		refresh 2000ms ·
failures		recent steps
no failures yet		09:17:47 ✓ session_cre
0 schema errors		09:20:53 ✓ agent_turn
0 circuit breaks		09:21:52 ✓ evaluator_r
0 agent errors		09:28:39 ✓ agent_turn
0 validator		09:30:42 ✓ agent_turn
regress		
0 integrity		
issues		
0 semantic		
failures		
0 ambiguity loops		

THE NEXT CHAPTER · Take 2 - Side B



avni-autopilot

Agentic workflows with Langraph

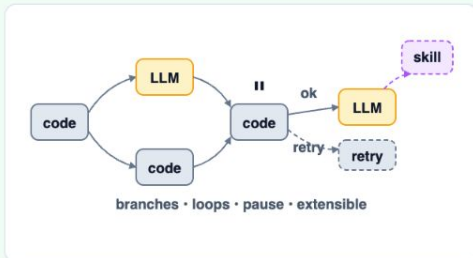
Orchestration framework choice

Three approaches, one shape that fits our pipeline.

OUR PICK

LangGraph

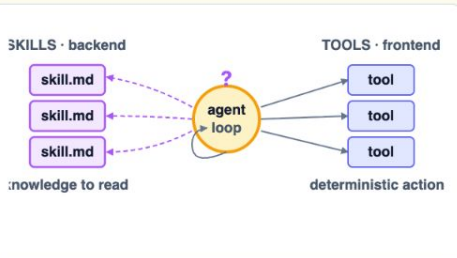
CODE-FIRST STATE MACHINE



- Mix **deterministic code** + LLM in one graph, branch + loop as needed
- **Skills can plug in** as nodes when useful — not forced, not locked out

Claude Agent SDK

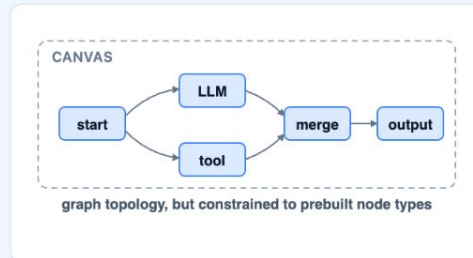
LLM-DRIVEN AGENT LOOP



- **Skills** are reusable knowledge docs (backend); **tools** are the user-facing actions (frontend)
- Using skills for **open-ended editing / fixing** is **non-deterministic** — the LLM interprets the skill on the fly, same prompt can pick different actions
- **Limited observability** — hard to tell which skill was picked or used

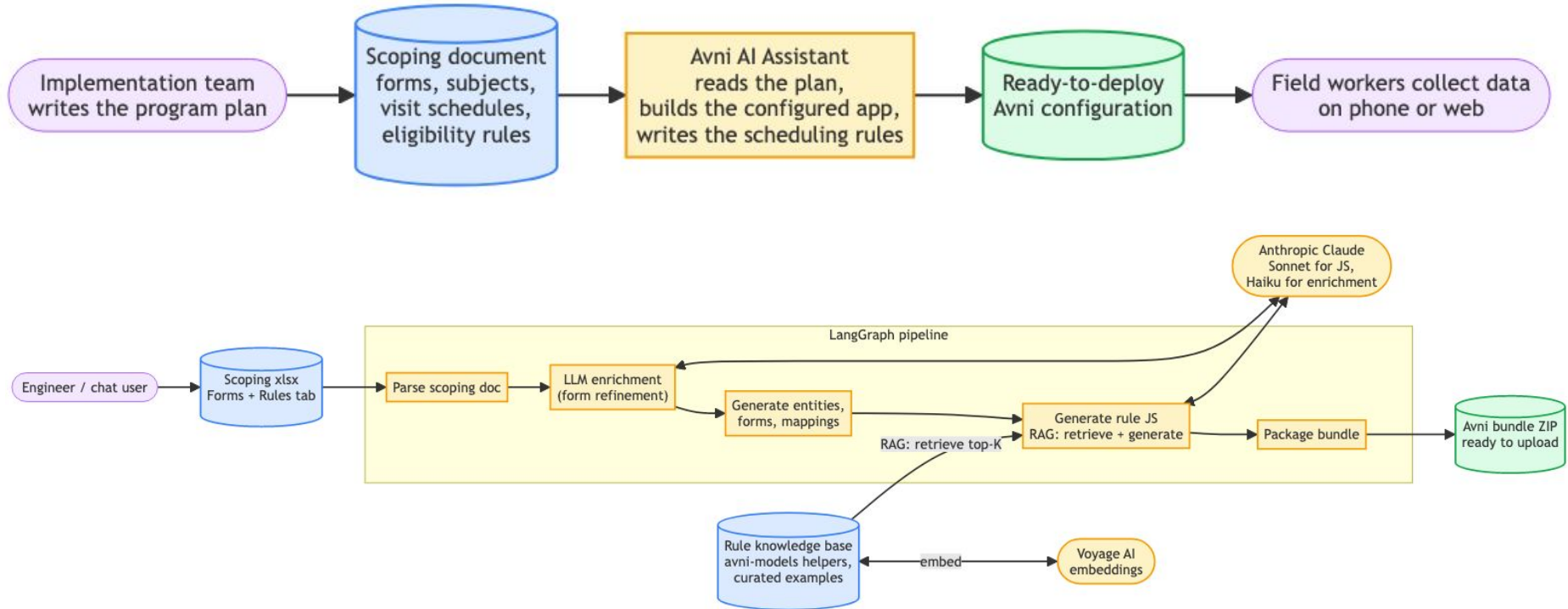
Dify

VISUAL LOW-CODE CANVAS



- Drag-and-drop visual nodes only — **difficult to design custom control flow** as needed
- Limited observability into anything that lives outside the canvas

Architecture diagram



Demonstration of the work done

The screenshot shows a web browser window with the URL `staging.avniproject.org/#/admin/upload`. The browser has several tabs open, including 'avni-autopilot - LangSmith', 'Copy of Durga India Modell...', 'Copy of Durga India Scoping', and 'Avni Web Console'. The page title is 'Upload' and the user is logged in as 'durga_india_ai_test8 (admin@durga_india_ai_test8)'. On the left, there is a sidebar menu with options: Location Types, Locations, Catchments, Users, User Groups, Upload (selected), Identifier Source, Identifier User Assign..., Languages, Organisation Details, and Phone Verification. The main content area has a 'Type' dropdown set to 'Metadata Zip', a 'CHOOSE FILE' button, and an 'UPLOAD' button. Below these is a 'Selected File:' section with a 'DOWNLOAD SAMPLE' link. To the right, there is instructional text: 'You can upload users, subjects, enrolments and encounters in bulk this screen. Metadata zip files that have been downloaded from another organisation can also be uploaded this screen. All files except the metadata zip are supposed to be in a CSV format. If you use Microsoft Excel, it has an option to save your spreadsheet in CSV format. Use the Download Sample option to download a sample file. More data about the sample file are available [here](#).' At the bottom, there is a table with columns: Execution Id, File name, Download Input, Type, Created at, Started at, Ended at, Status, Rows/Files Read, Rows/Files Completed, and Rows/Files. The table is currently empty. A 'REFRESH STATE' button is located at the bottom right of the table area. The macOS dock is visible at the bottom of the screen.

Upload

durga_india_ai_test8 (admin@durga_india_ai_test8)

Location Types

Locations

Catchments

Users

User Groups

Upload

Identifier Source

Identifier User Assign...

Languages

Organisation Details

Phone Verification

Type

Metadata Zip

Required

CHOOSE FILE

UPLOAD

Selected File:

DOWNLOAD SAMPLE

You can upload users, subjects, enrolments and encounters in bulk this screen. Metadata zip files that have been downloaded from another organisation can also be uploaded this screen.

All files except the metadata zip are supposed to be in a CSV format. If you use Microsoft Excel, it has an option to save your spreadsheet in CSV format.

Use the Download Sample option to download a sample file. More data about the sample file are available [here](#).

REFRESH STATE

Execution Id	File name	Download Input	Type	Created at	Started at	Ended at	Status	Rows/Files Read	Rows/Files Completed	Rows/Files
--------------	-----------	----------------	------	------------	------------	----------	--------	-----------------	----------------------	------------

Architectural learnings

What worked, picked deliberately.

LangGraph

FOR ORCHESTRATION

- **Observability** of every step in the run
- **Pause / resume** for human-in-the-loop review
- **Parallel + sequential** node execution side by side
- **Per-node retry** isolates failures
- **Chat-session state** persists across resumes

LLM vs deterministic

WHERE EACH ONE WINS

- Use the **LLM** when intelligent deduction is needed
- **Code it** when the path is clear and well-defined
- Never give the LLM **counting** tasks
- **Always validate** the LLM's output — syntax, grounding, schema

RAG

WHAT WE PICKED, WHAT MADE IT WORK

TECH CHOICES

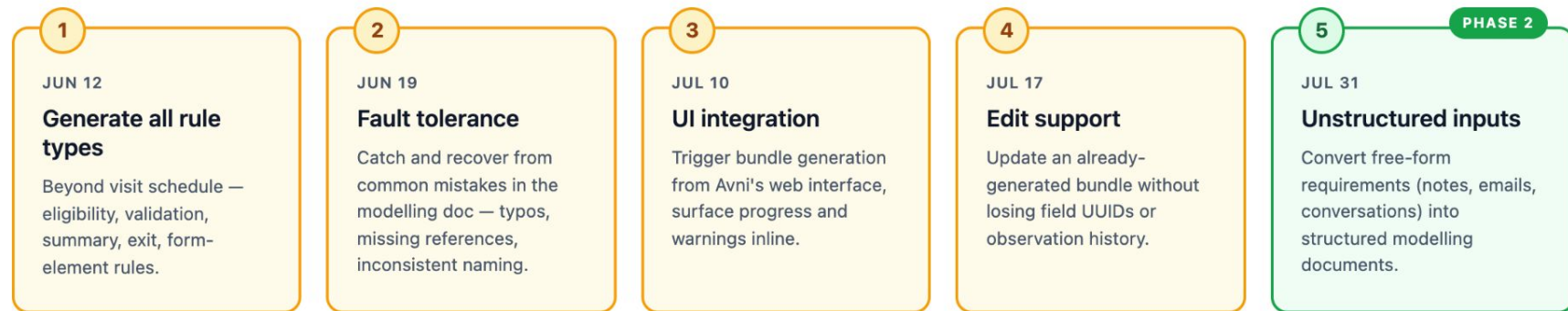
- Small corpus (<10K items): **flat JSON + brute-force cosine** is enough
- **Voyage AI embeddings** perform strongly on code-heavy retrieval
- **Content-hash invalidation + lazy embed** — zero API cost when nothing changed

DATA PREP MADE IT WORK

- **Curate ruthlessly** — dedupe + paired prompts beat raw dumps
- **Annotate every helper** (signature, use_when, snippet)
- Garbage in, garbage out — output mirrors corpus quality

Roadmap

Six weeks of focused build, then phase 2.



Expected outcome

Before vs. after targets across the two transformation phases.

PHASE 1

Requirements → Specification documents

1.5–2 days → ~0.5 day

Today	1.5–2 days of effort
-------	----------------------

Target	~0.5 day
--------	----------

Scope: AI automates the document creation. Discovery calls with the implementation team still happen — the AI doesn't replace them.

PHASE 2

Specification documents → Avni app

30% less effort

Today	100% of current effort
-------	------------------------

Target	~30% reduction
--------	----------------

Out of scope: field visits, online and offline reports (DB migration in progress), and project-management work — those continue at today's effort.

THANK YOU!